

**Mawlana Bhashani Science and Technology
University (MBSTU)**

Department of Information and Communication Technology (ICT)

Assignment Report

Section C Solutions

Course Title: Object Oriented Pattern Design (Java)

Course Code: ICT 2207

Submitted By:

Ashikur Rhaman Ashik
ID: IT-24047
Department of ICT
MBSTU

Submitted To:

Prof. Dr. Ziaur Rahman
Department of ICT
MBSTU

Date of Submission: June 27, 2026

*Mawlana Bhashani Science and Technology University
Santosh, Tangail-1902, Bangladesh*

Contents

1	Section C Solutions	2
1.1	Q1 (a) Conceptual Explanation	2
1.2	Q1 (b) Java Implementation / Analysis	2
1.3	Q2 (a) Conceptual Explanation	4
1.4	Q2 (b) Java Implementation / Analysis	4
1.5	Q3 (a) Conceptual Explanation	5
1.6	Q3 (b) Java Implementation / Analysis	6
1.7	Q4 (a) Conceptual Explanation	7
1.8	Q4 (b) Java Implementation / Analysis	8
1.9	Q5 (a) Conceptual Explanation	9
1.10	Q5 (b) Java Implementation / Analysis	9
2	Handwritten Work	12
2.1	Question 1: Abstract Class vs Interface (Complete Example)	12
2.2	Question 2: Static vs Non-Static Members & UML Class Diagram	15
2.3	Question 3: Constructor Overloading, Chaining, Method Overloading & Overriding	18

1 Section C Solutions

1.1 Q1 (a) Conceptual Explanation

Static Members:

- A **static member** (variable or method) belongs to the *class* itself rather than to any individual object.
- Static members are shared across all instances of the class — only one copy exists in memory regardless of how many objects are created.
- They can be accessed directly using the class name, without creating an object.
- Static variables are commonly used for counters, shared configuration values, or constants.
- Static methods cannot directly access non-static (instance) members, since they are not tied to a particular object.

Constructor Overloading:

- Constructor overloading means defining multiple constructors in the same class with different parameter lists (different number or type of parameters).
- It allows objects to be created in several different ways, increasing flexibility.

Constructor Chaining:

- Constructor chaining is the process of calling one constructor from another within the same class using `this(...)`, or calling a parent class constructor using `super(...)`.
- It helps avoid code duplication by reusing initialization logic defined in another constructor.
- `this()` or `super()` must be the **first statement** in the constructor body.

1.2 Q1 (b) Java Implementation / Analysis

The following example demonstrates a **Student** class with a static counter, overloaded constructors, and constructor chaining.

```
public class Student {  
    // Static member - shared by all objects  
    static int totalStudents = 0;  
  
    String name;  
    int id;  
    String department;  
}
```

```
// Constructor 1 - basic
public Student(String name, int id) {
    this(name, id, "Not Assigned"); // constructor chaining
}

// Constructor 2 - full (overloaded constructor)
public Student(String name, int id, String department) {
    this.name = name;
    this.id = id;
    this.department = department;
    totalStudents++; // increment static counter
}

public void display() {
    System.out.println("Name: " + name + ", ID: " + id +
        ", Department: " + department);
}

public static void showTotal() {
    System.out.println("Total Students Created: " + totalStudents);
}

public static void main(String[] args) {
    Student s1 = new Student("Ashik", 101, "ICT");
    Student s2 = new Student("Rafi", 102);

    s1.display();
    s2.display();
    Student.showTotal();
}
}
```

Expected Output:

```
Name: Ashik, ID: 101, Department: ICT
Name: Rafi, ID: 102, Department: Not Assigned
Total Students Created: 2
```

Explanation of Flow:

- When `s2` is created using the 2-argument constructor, it calls `this(name, id, "Not Assigned")`, which chains to the 3-argument constructor.
- The static variable `totalStudents` is incremented only once per actual object construction and is shared by the entire class.

1.3 Q2 (a) Conceptual Explanation

Abstract Class vs Interface:

- An **abstract class** is a class declared with the **abstract** keyword that may contain both abstract methods (no body) and concrete methods (with implementation).
- An **interface** traditionally contains only abstract method declarations (Java 8+ also allows **default** and **static** methods with bodies).
- A class can extend only **one** abstract class (single inheritance) but can implement **multiple** interfaces.
- Abstract classes can have constructors, instance variables, and access modifiers (private, protected); interfaces cannot have constructors, and all fields are implicitly **public static final**.
- Use an abstract class when classes share a common base implementation; use an interface when unrelated classes need to guarantee a common behavior/contract.

Comparison Table:

Feature	Abstract Class	Interface
Methods	Abstract + Concrete	Abstract (+ default/static in Java 8+)
Inheritance	Single (extends)	Multiple (implements)
Constructors	Allowed	Not Allowed
Variables	Any type	public static final only
Access Modifiers	All allowed	Public by default

1.4 Q2 (b) Java Implementation / Analysis

```
// Abstract class
abstract class Shape {
    String color;

    Shape(String color) {
        this.color = color;
    }

    abstract double area(); // abstract method

    void displayColor() { // concrete method
        System.out.println("Shape Color: " + color);
    }
}

// Interface
```

```
interface Drawable {
    void draw(); // implicitly public abstract
}

// Concrete class using both
class Circle extends Shape implements Drawable {
    double radius;

    Circle(String color, double radius) {
        super(color);
        this.radius = radius;
    }

    @Override
    double area() {
        return Math.PI * radius * radius;
    }

    @Override
    public void draw() {
        System.out.println("Drawing a Circle of radius " + radius);
    }

    public static void main(String[] args) {
        Circle c = new Circle("Red", 5.0);
        c.displayColor();
        c.draw();
        System.out.println("Area: " + c.area());
    }
}
```

Expected Output:

```
Shape Color: Red
Drawing a Circle of radius 5.0
Area: 78.53981633974483
```

1.5 Q3 (a) Conceptual Explanation

Overloading vs Overriding:

- **Method Overloading** occurs when two or more methods in the *same class* share the same name but differ in parameter type, number, or order.
- **Method Overriding** occurs when a subclass provides a specific implementation of a method already defined in its superclass, with the *same signature*.

- Overloading is resolved at **compile-time** (also called static/early binding) because the compiler decides which method to call based on the method signature available at compile time.
- Overriding is resolved at **run-time** (also called dynamic/late binding) because the JVM decides which method to invoke based on the actual object type during execution.

Key Differences:

Aspect	Overloading	Overriding
Class Involved	Same class	Parent-Child classes
Signature	Must differ	Must be identical
Binding Type	Compile-time (static)	Run-time (dynamic)
Return Type	Can differ	Must be same/covariant
Purpose	Increase readability/flexibility	Achieve runtime polymorphism

1.6 Q3 (b) Java Implementation / Analysis

```
class Calculator {
    // Method Overloading - compile time polymorphism
    int add(int a, int b) {
        return a + b;
    }

    double add(double a, double b) {
        return a + b;
    }

    int add(int a, int b, int c) {
        return a + b + c;
    }
}

class Animal {
    void sound() {
        System.out.println("Animal makes a sound");
    }
}

class Dog extends Animal {
    // Method Overriding - runtime polymorphism
    @Override
    void sound() {
        System.out.println("Dog barks");
    }
}
```

```
public class TestDemo {
    public static void main(String[] args) {
        // Overloading demonstration
        Calculator calc = new Calculator();
        System.out.println(calc.add(2, 3));           // calls int version
        System.out.println(calc.add(2.5, 3.5));       // calls double version
        System.out.println(calc.add(1, 2, 3));        // calls 3-arg version

        // Overriding demonstration
        Animal a = new Dog(); // Reference type: Animal, Object type: Dog
        a.sound();             // Decided at RUNTIME -> "Dog barks"
    }
}
```

Expected Output:

```
5
6.0
6
Dog barks
```

Step-by-Step Reasoning:

1. For `calc.add(2, 3)`, the compiler checks argument types at **compile-time** and selects the `int add(int, int)` method — this is overloading.
2. For `a.sound()`, although the reference type is `Animal`, the actual object created is `Dog`.
3. The JVM looks up the method table of the actual object (`Dog`) at **run-time** and executes `Dog`'s overridden `sound()` method.
4. This demonstrates dynamic method dispatch — the core mechanism of runtime polymorphism.

1.7 Q4 (a) Conceptual Explanation

The Object Class and toString():

- In Java, every class implicitly extends the `Object` class if it does not explicitly extend another class — making `Object` the root of the entire class hierarchy.
- The `Object` class provides several default methods, including `toString()`, `equals()`, `hashCode()`, `getClass()`, etc.
- The default `toString()` method returns a string in the format: `ClassName@HexHashCode`, which is rarely useful for debugging or display.
- By overriding `toString()`, a class can return a meaningful, human-readable representation of its object's state.

- `toString()` is automatically invoked when an object is printed using `System.out.println()` or concatenated with a `String`.

1.8 Q4 (b) Java Implementation / Analysis

```
class Book {
    String title;
    String author;
    double price;

    Book(String title, String author, double price) {
        this.title = title;
        this.author = author;
        this.price = price;
    }
    // No toString() override here - uses Object's default
}

class BookWithToString {
    String title;
    String author;
    double price;

    BookWithToString(String title, String author, double price) {
        this.title = title;
        this.author = author;
        this.price = price;
    }

    // Overriding Object's toString() method
    @Override
    public String toString() {
        return "Book[Title=" + title + ", Author=" + author +
            ", Price=" + price + "]";
    }
}

public class ObjectDemo {
    public static void main(String[] args) {
        Book b1 = new Book("Data Structures", "Lafore", 450.0);
        System.out.println(b1); // uses Object's default toString()

        BookWithToString b2 =
            new BookWithToString("Design Patterns", "GoF", 650.0);
        System.out.println(b2); // uses overridden toString()
    }
}
```

Expected Output (sample):

Book@1b6d3586

Book[Title=Design Patterns, Author=GoF, Price=650.0]

Note: The hash-code portion of the default output (@1b6d3586) will vary on each run, since it is derived from the object's memory-based hash code.

1.9 Q5 (a) Conceptual Explanation

Polymorphism and Dynamic Binding:

- **Polymorphism** means "many forms" — the ability of an object to take on multiple behaviors depending on context.
- **Compile-time Polymorphism** (static binding) is achieved through method overloading; the method to call is determined by the compiler.
- **Run-time Polymorphism** (dynamic binding) is achieved through method overriding; the method to call is determined by the JVM based on the actual object at runtime.
- **Dynamic Binding** works through a mechanism called *dynamic method dispatch*: a superclass reference variable can hold a subclass object, and at runtime the JVM invokes the method version belonging to the actual (subclass) object.
- This allows writing flexible, extensible code where new subclasses can be added without modifying existing logic — a key OOP design principle (Open/Closed Principle).

1.10 Q5 (b) Java Implementation / Analysis

```
abstract class Employee {
    String name;

    Employee(String name) {
        this.name = name;
    }

    abstract double calculateSalary();
}

class FullTimeEmployee extends Employee {
    double monthlySalary;

    FullTimeEmployee(String name, double monthlySalary) {
        super(name);
        this.monthlySalary = monthlySalary;
    }
}
```

```
@Override
double calculateSalary() {
    return monthlySalary;
}

class PartTimeEmployee extends Employee {
    double hourlyRate;
    int hoursWorked;

    PartTimeEmployee(String name, double hourlyRate, int hoursWorked) {
        super(name);
        this.hourlyRate = hourlyRate;
        this.hoursWorked = hoursWorked;
    }

    @Override
    double calculateSalary() {
        return hourlyRate * hoursWorked;
    }
}

public class PolymorphismDemo {
    public static void main(String[] args) {
        Employee[] staff = {
            new FullTimeEmployee("Ashik", 35000),
            new PartTimeEmployee("Rafi", 500, 80)
        };

        // Dynamic binding in action
        for (Employee e : staff) {
            System.out.println(e.name + " Salary: " + e.calculateSalary());
        }
    }
}
```

Expected Output:

```
Ashik Salary: 35000.0
Rafi Salary: 40000.0
```

Step-by-Step Reasoning:

1. The array `staff` is declared with the type `Employee[]`, but holds objects of `FullTimeEmployee` and `PartTimeEmployee`.
2. During each loop iteration, the compiler only verifies that `calculateSalary()` exists in the `Employee` type — this check is **compile-time**.

3. At **run-time**, the JVM examines the actual object referenced by `e` and dynamically dispatches the call to the correct overridden method.
4. This is dynamic binding: identical method calls (`e.calculateSalary()`) produce different results depending on the real object type, demonstrating true runtime polymorphism.

2 Handwritten Work

The following sub-sections contain the handwritten solutions corresponding to the theoretical and code-based answers of the three practice questions.

2.1 Question 1: Abstract Class vs Interface (Complete Example)

When to choose what:

- Use an **Abstract Class** when related classes share a common base state/behavior (a clear "is-a" relationship with shared code).
- Use an **Interface** when unrelated classes must guarantee the same capability/contract, regardless of their inheritance hierarchy ("can-do" relationship).
- Rule of thumb: *Abstract class = shared identity & code reuse; Interface = shared capability & multiple inheritance of behavior.*

Scenario: `Vehicle` is a natural abstract class (all vehicles share fields like `speed`, `brand`, and common logic like `startEngine()`). `Chargeable` is an interface, because completely unrelated classes (a `Car` and a `Mobile`) may both need charging capability, but they do not share a common parent.

```
// Abstract class - shared state & partial implementation (is-a)
abstract class Vehicle {
    String brand;
    int speed;

    Vehicle(String brand, int speed) {
        this.brand = brand;
        this.speed = speed;
    }

    void startEngine() {                // shared concrete behavior
        System.out.println(brand + " engine started.");
    }

    abstract void move();                // must be implemented differently
}

// Interface - shared capability across unrelated classes (can-do)
interface Chargeable {
    void chargeBattery();                // contract only
}

// Car IS-A Vehicle, and CAN-BE Chargeable
class ElectricCar extends Vehicle implements Chargeable {
```

```
ElectricCar(String brand, int speed) {
    super(brand, speed);
}

@Override
void move() {
    System.out.println(brand + " car is driving at " + speed + " km/h.");
}

@Override
public void chargeBattery() {
    System.out.println(brand + " car is charging via charging station.");
}
}

// Mobile is NOT a Vehicle, but it CAN-BE Chargeable too
class Mobile implements Chargeable {
    String model;

    Mobile(String model) {
        this.model = model;
    }

    @Override
    public void chargeBattery() {
        System.out.println(model + " mobile is charging via USB cable.");
    }
}

public class AbstractVsInterfaceDemo {
    public static void main(String[] args) {
        ElectricCar car = new ElectricCar("Tesla", 180);
        car.startEngine();
        car.move();
        car.chargeBattery();

        Mobile phone = new Mobile("Samsung");
        phone.chargeBattery();
    }
}
```

Expected Output:

```
Tesla engine started.
Tesla car is driving at 180 km/h.
Tesla car is charging via charging station.
Samsung mobile is charging via USB cable.
```

Justification:

- **Vehicle** is abstract because **ElectricCar** (and any future **Bike**, **Truck**, etc.) genuinely **is a Vehicle** and reuses **startEngine()**.
- **Chargeable** is an interface because **ElectricCar** and **Mobile** have no common parent, yet both need the **same capability** **contra ct**.

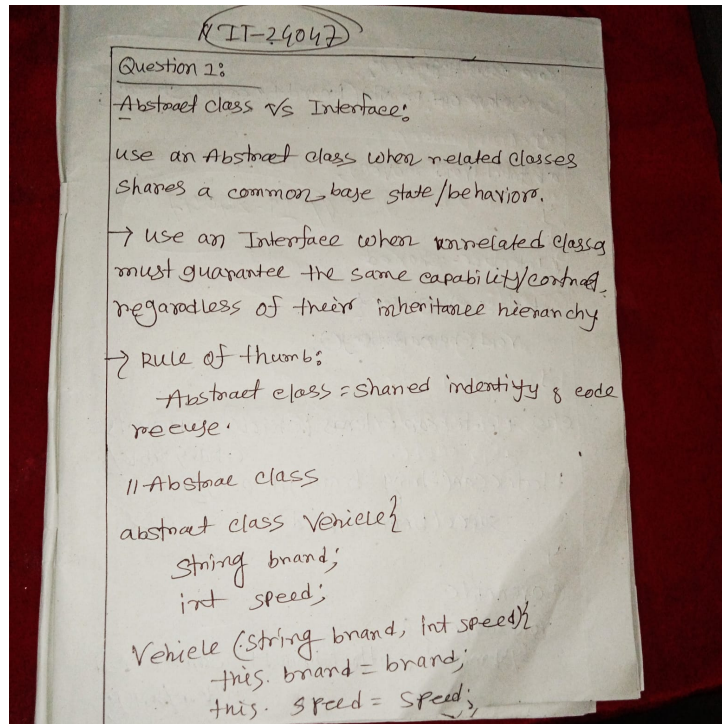


Figure 1: Handwritten Solution - Question 1 (Abstract Class vs Interface)

2.2 Question 2: Static vs Non-Static Members & UML Class Diagram

Difference between Static and Non-Static Members:

Aspect	Static Member	Non-Static (Instance) Member
Belongs to	The class	Each individual object
Memory	One copy shared by all objects	Separate copy per object
Access	ClassName.member or directly inside class	object.member
Lifetime	Exists as long as the class is loaded	Exists as long as the object exists
Access from static method	Directly accessible	Not directly accessible (needs an object)
Typical use	Counters, constants, utility methods	Object-specific data (name, id, etc.)

UML Class Diagram (Text-Based Representation):

UML notation: + = public, - = private, underlined = static member.

```

+-----+
|           BankAccount           |
+-----+
| - accountHolder : String         |
| - balance : double               |
| - accountNumber : int            |
| -- bankName : String__ (static) |
| -- totalAccounts : int__ (static) |
+-----+
| + BankAccount(holder: String, bal: double) |
| + deposit(amount: double) : void           |
| + withdraw(amount: double) : void          |
| + getBalance() : double                    |
| ++ getTotalAccounts() : int__ (static) |
| ++ main(args: String[]) : void__ (static) |
+-----+

```

(Note: underlined entries indicate static members per standard UML convention; in handwritten form, draw an underline under *bankName*, *totalAccounts*, *getTotalAccounts()*, and *main()*.)

Corresponding Java Code:

```

public class BankAccount {
    // Non-static (instance) fields - private

```



```
private String accountHolder;
private double balance;
private int accountNumber;

// Static fields - private, shared across all objects
private static String bankName = "MBSTU Bank";
private static int totalAccounts = 0;

public BankAccount(String accountHolder, double balance) {
    this.accountHolder = accountHolder;
    this.balance = balance;
    this.accountNumber = ++totalAccounts; // auto-generated ID
}

// Non-static (instance) public methods
public void deposit(double amount) {
    balance += amount;
}

public void withdraw(double amount) {
    if (amount <= balance) {
        balance -= amount;
    } else {
        System.out.println("Insufficient balance!");
    }
}

public double getBalance() {
    return balance;
}

// Static public method
public static int getTotalAccounts() {
    return totalAccounts;
}

public static void main(String[] args) {
    BankAccount acc1 = new BankAccount("Ashik", 5000);
    BankAccount acc2 = new BankAccount("Rafi", 3000);

    acc1.deposit(1500);
    acc2.withdraw(500);

    System.out.println("Acc1 Balance: " + acc1.getBalance());
    System.out.println("Acc2 Balance: " + acc2.getBalance());
    System.out.println("Total Accounts: " + BankAccount.getTotalAccounts());
}
}
```

Expected Output:

Acc1 Balance: 6500.0

Acc2 Balance: 2500.0

Total Accounts: 2

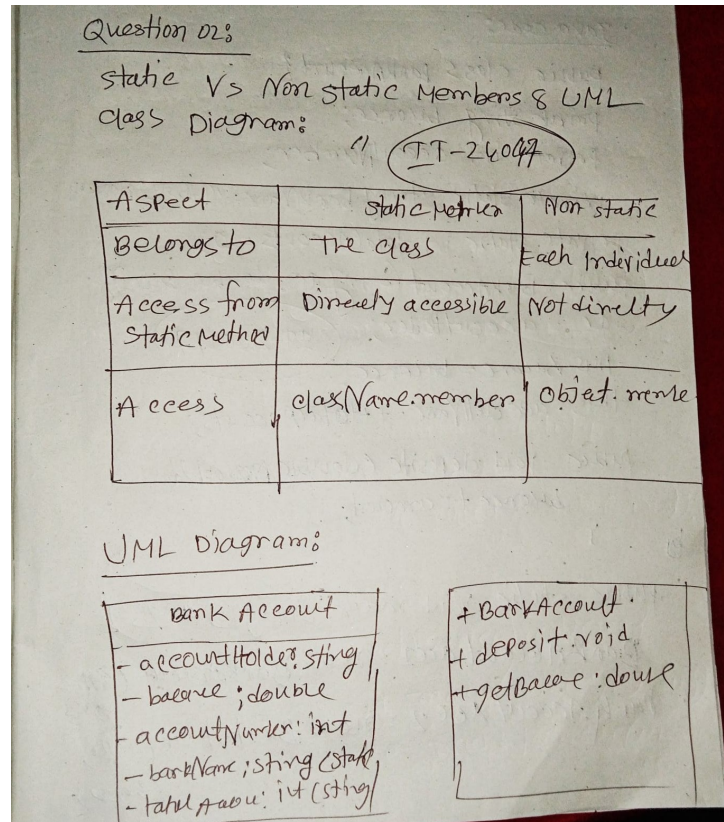


Figure 2: Handwritten Solution - Question 2 (Static vs Non-Static & UML Diagram)

2.3 Question 3: Constructor Overloading, Chaining, Method Overloading & Overriding

Example 1: Person → Student (Inheritance Hierarchy)

```
class Person {
    String name;
    int age;

    // Constructor Overloading (within Person class)
    Person() {
        this("Unknown", 0);           // Constructor Chaining (this)
    }

    Person(String name, int age) {
        this.name = name;
        this.age = age;
    }

    // Method Overloading (within Person class)
    void display() {
        System.out.println("Name: " + name + ", Age: " + age);
    }

    void display(String prefix) {
        System.out.println(prefix + " -> Name: " + name + ", Age: " + age);
    }
}

class Student extends Person {
    String studentId;

    Student(String name, int age, String studentId) {
        super(name, age);             // Constructor Chaining (super)
        this.studentId = studentId;
    }

    // Method Overriding (Student overrides Person's display())
    @Override
    void display() {
        System.out.println("Student -> Name: " + name +
                           ", Age: " + age + ", ID: " + studentId);
    }
}

public class InheritanceDemo1 {
    public static void main(String[] args) {
        Person p1 = new Person();      // chains to Person(String, int)
    }
}
```

```
p1.display();                // overloaded display()
p1.display("Person Info");    // overloaded display(String)

Student s1 = new Student("Ashik", 22, "IT-24047");
s1.display();                // overridden display() runs
    }
}
```

Expected Output:

```
Name: Unknown, Age: 0
Person Info -> Name: Unknown, Age: 0
Student -> Name: Ashik, Age: 22, ID: IT-24047
```

Example 2: Shape → Rectangle (Inheritance Hierarchy)

```
class Shape {
    String color;

    // Constructor Overloading (within Shape class)
    Shape() {
        this("White");                // Constructor Chaining (this)
    }

    Shape(String color) {
        this.color = color;
    }

    // Method Overloading (within Shape class)
    double area() {
        return 0.0;
    }

    double area(double side) {
        return side * side;          // treats shape as a square
    }
}

class Rectangle extends Shape {
    double length;
    double width;

    Rectangle(String color, double length, double width) {
        super(color);                // Constructor Chaining (super)
        this.length = length;
        this.width = width;
    }
}
```

```
// Method Overriding (Rectangle overrides Shape's area())
@Override
double area() {
    return length * width;
}
}

public class InheritanceDemo2 {
    public static void main(String[] args) {
        Shape s = new Shape();           // chains to Shape(String)
        System.out.println("Default Shape Color: " + s.color);
        System.out.println("Square Area (side=4): " + s.area(4));    // overloaded

        Rectangle r = new Rectangle("Blue", 5, 3);
        System.out.println("Rectangle Color: " + r.color);
        System.out.println("Rectangle Area: " + r.area());           // overridden
    }
}
```

Expected Output:

```
Default Shape Color: White
Square Area (side=4): 16.0
Rectangle Color: Blue
Rectangle Area: 15.0
```

Summary of Concepts Demonstrated:

- **Constructor Overloading:** Multiple constructors in `Person` and `Shape` with different parameter lists.
- **Constructor Chaining:** `this(...)` used to chain within the same class; `super(...)` used to chain to the parent class constructor.
- **Method Overloading:** `display()/display(String)` and `area()/area(double)` — same method name, different parameters, resolved at **compile-time**.
- **Method Overriding:** `Student` and `Rectangle` redefine the parent's method with the same signature, resolved at **run-time** via dynamic binding.

```

void display() {
    System.out.println(prefix + " -> Name: " +
        name + ", Age: " + age);
}

class student extends person {
    String studentId;
    student(String name, int age, String studentId) {
        super(name, age);
        this.studentId = studentId;
    }
    // Method overriding
    @Override
    void display() {
        System.out.println("Student -> Name: " + name +
            ", Age: " + age + ", ID: " +
            studentId);
    }
}

```

Figure 3: Handwritten Solution - Question 3 (Constructors & Method Overloading/Overriding) - Page 1

```

void display() {
    System.out.println(prefix + " -> Name: " +
        name + ", Age: " + age);
}

class student extends person {
    String studentId;
    student(String name, int age, String studentId) {
        super(name, age);
        this.studentId = studentId;
    }
    // Method overriding
    @Override
    void display() {
        System.out.println("Student -> Name: " + name +
            ", Age: " + age + ", ID: " +
            studentId);
    }
}

```

Figure 4: Handwritten Solution - Question 3 (Constructors & Method Overloading/Overriding) - Page 2

Question 1:

Abstract class vs Interface:

use an Abstract class when related classes share a common base state/behavior.

→ use an Interface when unrelated classes must guarantee the same capability/contract, regardless of their inheritance hierarchy.

→ Rule of thumb:

Abstract class = shared identity & code reuse.

// Abstract class

```
abstract class Vehicle {
```

```
    String brand;
```

```
    int speed;
```

```
Vehicle (String brand, int speed) {
```

```
    this.brand = brand;
```

```
    this.speed = speed;
```

}
abstract void move();
}

//IT-24047

//Interface - Shared

interface Chargeable {
 void ChargeBattery();
}

Class ElectricCar extends Vehicle implements Chargeable

ElectricCar(String brand, int speed) {
 super(brand, speed);
}

@Override

void move() {
 System.out.println(brand + " car " + speed
 + " km/h.");
}


```
car.startEngine();  
car.move();  
car.charBattery();  
Mobile phone = new Mobile("Samsung");  
phone.charBattery();  
}
```

IT-24047

Output:

tesla engine started.

tesla car is driving at 180 km/h.

Samsung mobile is charging via
USB cable.

Question 02:

Static Vs Non static Members & UML
class Diagram:

// IT-26047

Aspect	Static Member	Non static
Belongs to	The class	Each individual
Access from Static Method	Directly accessible	Not directly
Access	className.member	Object.member

UML Diagram:

Bank Account
- accountHolder: string
- balance: double
- accountNumber: int
- bankName: string (static)
- totalAcount: int (string)

```
+ BankAccount  
+ deposit: void  
+ getBalance: double
```

```

public class BankAccount {
    private String balance;
    private int accountNumber;
    private static String bankName = "MBSTU Bank";
    private static int totalAccounts = 0;
    public BankAccount(String ac, double bal) {
        - ac = accountNumber;
        - this.balance = balance;
        - this.accountName = ++totalAccounts;
    }
    public void deposit (double amount) {
        balance += amount;
    }
}

```

```

public static void main (String[] args) {
    BankAccount acc1 = new BankAccount ("123456", 1000);
    BankAccount acc2 = new BankAccount ("789", 500);
}

```


getBalance();

System.out.println("Ace2 Balance: " + ace2.getBalance());

IT-24047

output:

Ace1 Balance: 5500.0

Ace2 Balance: 2500.0

total accounts: 2

Overloading & overriding
Overloading → chaining → Method

Example 1: Person → Student

```
class Person {
```

```
    String Name;
```

```
    int age;
```

```
    Person() {
```

```
        this("unknown", 0);
```

```
    Person(Stringname name, int age) {
```

```
        this.name = name;
```

```
        this.age = age;
```

```
    void display() {
```

```
        System.out.println("Name: " + name + "  
Age: " + age);
```



```
System.out.println(prefix + " -> Name: " +  
name + ", Age: " + age);
```

```
}  
}
```

```
class student extends person {  
    String studentId;
```

```
    student(String name, int age, String stud  
        super(name, age);
```

```
    this.studentId = studentId;
```

```
}
```

```
// Method Overriding
```

```
@Override
```

```
void display () {
```

```
    System.out.println("Student -> Name: " + name +  
        ", Age: " + age + ", ID: " +
```

```
    studentId);
```

```
public static void main(String[] args)
```

```
Person P1 = new Person();
```

```
P1.display();
```

```
P1.display("Person Info");
```

Output:

Name: Unknown, Age: 0.

Person Info → Name: Unknown, Age: 0

Student → Name: Ashik, Age: 22, ID:

IT-20047